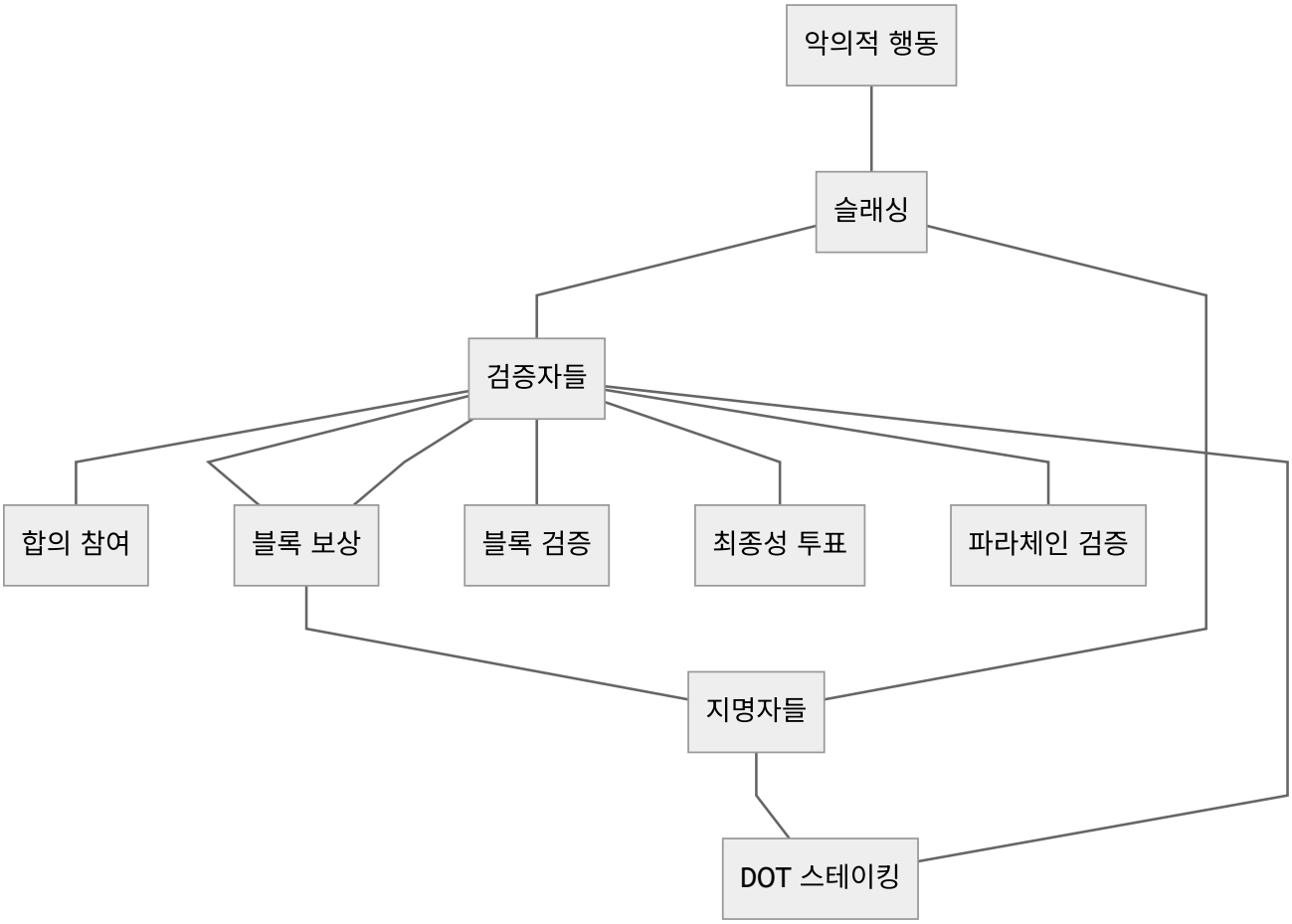
3.2 검증자 시스템 및 스테이킹 메커니즘 분석

개요

Polkadot의 검증자 시스템은 네트워크 보안과 합의를 담당하는 핵심 구성 요소입니다. 이 시스템은 Nominated Proof-of-Stake (NPoS) 합의 메커니즘을 기반으로 하며, 검증자(Validator)와 지명자(Nominator)가 협력하여 네트워크를 보호합니다. 이 문서에서는 검증자 노드 설정, 스테이킹 메커니즘, 보상 시스템, 슬래싱 조건을 상세히 분석합니다.

검증자 시스템 아키텍처

NPoS (Nominated Proof-of-Stake) 개요



검증자 역할과 책임

주요 역할:

1. **블록 생성**: BABE 알고리즘을 통한 블록 생성
2. **최종성 투표**: GRANDPA를 통한 블록 최종성 결정
3. **파라체인 검증**: 파라체인 블록 검증 및 가용성 보장
4. **네트워크 보안**: 악의적 행동 감지 및 신고

검증자 노드 설정 및 운영 방법

1. 하드웨어 요구사항

최소 요구사항:

- **CPU**: 4코어 이상 (Intel/AMD x86-64)
- **메모리**: 16GB RAM (권장 32GB)
- **스토리지**: 1TB NVMe SSD (확장 가능)
- **네트워크**: 1Gbps 대역폭, 고정 IP

권장 사양:

- **CPU**: 8코어 이상 고성능 프로세서
- **메모리**: 64GB RAM
- **스토리지**: 2TB+ NVMe SSD (RAID 구성)
- **네트워크**: 10Gbps 대역폭, 이중화 연결

2. 검증자 노드 설치 및 설정

기본 설치

```
# Polkadot 바이너리 다운로드
wget https://github.com/paritytech/polkadot-sdk/releases/latest/download/polkad
chmod +x polkadot
sudo mv polkadot /usr/local/bin/

# 워커 바이너리도 함께 설치
wget https://github.com/paritytech/polkadot-sdk/releases/latest/download/polkad
wget https://github.com/paritytech/polkadot-sdk/releases/latest/download/polkad
chmod +x polkadot-*-worker
```

```
sudo mv polkadot-*-worker /usr/local/bin/
```

검증자 키 생성

```
# 세션 키 생성
polkadot key generate --scheme sr25519 --output-type json

# 스테이킹 키 생성 (별도 보안 환경에서)
polkadot key generate --scheme sr25519 --output-type json --words 24

# 키 삽입 (노드 실행 중)
curl -H "Content-Type: application/json" -d '{"id":1, "jsonrpc":"2.0", "meth
curl -H "Content-Type: application/json" -d '{"id":1, "jsonrpc":"2.0", "meth
curl -H "Content-Type: application/json" -d '{"id":1, "jsonrpc":"2.0", "meth
curl -H "Content-Type: application/json" -d '{"id":1, "jsonrpc":"2.0", "meth
curl -H "Content-Type: application/json" -d '{"id":1, "jsonrpc":"2.0", "meth
curl -H "Content-Type: application/json" -d '{"id":1, "jsonrpc":"2.0", "meth
```

검증자 노드 실행

```
# 기본 검증자 노드 실행
polkadot \
--validator \
```

```
--name  "MyValidator" \
--chain polkadot \
--base-path  /var/lib/polkadot \
--port 30333 \
--rpc-port  9933 \
--ws-port 9944 \
--prometheus-port  9615 \
--prometheus-external \
--telemetry-url  'wss://telemetry.polkadot.io/submit/ 0'
```

3. 보안 검증자 모드 설정

```
# 보안 검증자 모드 (기본값)
polkadot \
--validator \
--secure-validator-mode \
--name "SecureValidator" \
--chain  polkadot \
--base-path /var/lib/polkadot \
--rpc-methods  Safe \
--no-unsafe-rpc-external
```

보안 검증자 모드 특징:

- RPC 외부 접근 차단

- 안전하지 않은 RPC 메서드 비활성화
- 텔레메트리만 외부 연결 허용
- 워커 프로세스 격리 강화

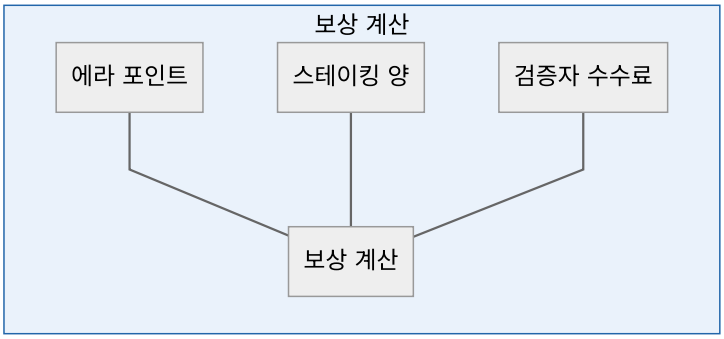
4. systemd 서비스 설정

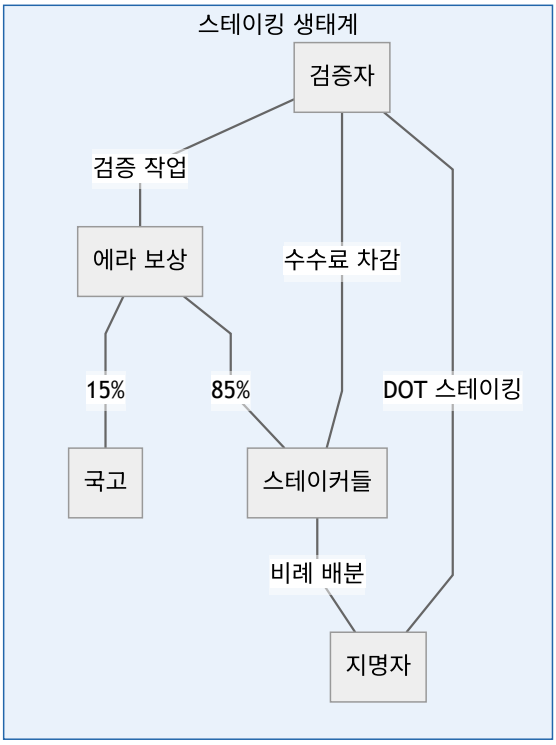
```
# /etc/systemd/system/polkadot-validator.service
[Unit]
Description=Polkadot Validator
After=network.target
StartLimitIntervalSec=0

[Service]
Type=simple
Restart=always
RestartSec=10
User=polkadot
SyslogIdentifier=polkadot
SyslogFacility=local7
KillSignal=SIGHUP
ExecStart=/usr/local/bin/polkadot \
--validator \
--secure-validator-mode \
--name "MyValidator" \
```

```
--chain polkadot \

--base-path /var/lib/polkadot \

--prometheus-external \

--prometheus-port 9615 \

--telemetry-url 'wss://telemetry.polkadot.io/submit/ 0'



[Install]

WantedBy=multi-user.target
```

스테이킹 시스템 구조 및 보상 메커니즘

스테이킹 시스템 아키텍처





스테이킹 매개변수 (Westend 기준)

```
// 에라 및 세션 설정
pub const SessionsPerEra: SessionIndex = 6; // 에라당 6세션 (6시간)
pub const BondingDuration: EraIndex = 2; // 언본딩 기간 2에라 (12시간)
pub const SlashDeferDuration: EraIndex = 1; // 슬래시 지연 1에라 (6시간)

// 스테이킹 제한
pub const MaxNominators: u32 = 64; // 최대 지명자 수
pub const MaxNominations: u32 = 16; // 지명자당 최대 지명 수
```

```
pub const MaxExposurePageSize: u32 = 64; // 노출 페이지 크기
```

보상 계산 메커니즘

1. 에라 보상 계산

```
impl pallet_staking::EraPayout<Balance> for EraPayout {
fn era_payout(
_total_staked: Balance,
_total_issuance: Balance,
era_duration_millis: u64,
) -> (Balance, Balance) {
const MILLISECONDS_PER_YEAR: u64 = (1000 * 3600 * 24 * 36525) / 100;
// 에라 길이 비율 계산 (1년 대비)
let relative_era_len = FixedU128::from_rational(
era_duration_millis.into(),
MILLISECONDS_PER_YEAR.into()
);


// 고정 총 발행량 기준 (Westend)
let fixed_total_issuance: i128 = 5_216_342_402_773_185_773;
let fixed_inflation_rate = FixedU128::from_rational(8, 100); // 8% 연간 인플레이션
let yearly_emission = fixed_inflation_rate.saturating_mul_int(fixed_total_issua
let era_emission = relative_era_len.saturating_mul_int(yearly_emission);
```

```
// 15% 국고, 85% 스테이커
let to_treasury = FixedU128::from_rational(15, 100).saturating_mul_int(era_emis
let to_stakers = era_emission.saturating_sub(to_treasury);


(to_stakers.saturated_into(), to_treasury.saturated_into())
}
}
```

2. 개별 보상 분배

검증자 보상 계산:

```
검증자_보상 = (에라_포인트 / 총_에라_포인트) × 총_스테이커_보상 × (1 - 수수료율)
수수료_보상 = (에라_포인트 / 총_에라_포인트) × 총_스테이커_보상 × 수수료율
```

지명자 보상 계산:

```
지명자_보상 = 검증자_보상 × (지명자_스테이킹 / 검증자_총_스테이킹)
```

스테이킹 라이프사이클

1. 본딩 (Bonding)

```
# 스테이킹 본딩
polkadot-js-api tx.staking.bond(controller, value, payee)
```

```
# 추가 본딩
polkadot-js-api  tx.staking.bondExtra(maxAdditional)
```

2. 지명 (Nominating)

```
# 검증자 지명
polkadot-js-api  tx.staking.nominate([validator1, validator2, ...])

# 지명 변경
polkadot-js-api  tx.staking.nominate([newValidator1, newValidator2, ...])
```

3. 언본딩 (Unbonding)

```
# 일부 언본딩
polkadot-js-api  tx.staking.unbond(value)

# 완전 언본딩
polkadot-js-api  tx.staking.chill()
polkadot-js-api  tx.staking.unbond(allStaked)
```

4. 출금 (Withdrawal)

```
# 언본딩된 자금 출금
polkadot-js-api  tx.staking.withdrawUnbonded(numSlashingSpans)
```

슬래싱 조건 및 패널티 시스템

슬래싱 가능한 위반 행위

1. GRANDPA 위반

Equivocation (이중 투표):

- 같은 라운드에서 서로 다른 블록에 투표
- **패널티**: 스테이킹의 1/10000 슬래시

```
// GRANDPA 이중 투표 감지
pub struct GrandpaEquivocation {
pub round_number: u64,
pub identity: AuthorityId,
pub first: (Precommit, Signature),
pub second: (Precommit, Signature),
}
```

2. BABE 위반

Equivocation (이중 생성):

- 같은 슬롯에서 여러 블록 생성
- **패널티**: 스테이킹의 1/10000 슬래시

```
// BABE 이중 생성 감지
pub struct BabeEquivocation {
pub slot: Slot,
```

```
pub offender: AuthorityId,
pub first_header: Header,
pub second_header: Header,
}
```

3. 파라체인 관련 위반

Invalid Backing:

- 잘못된 파라체인 블록 지지
- **패널티**: 스테이킹의 1/1000 슬래시

Invalid Availability:

- 가용성 데이터 제공 실패
- **패널티**: 스테이킹의 1/1000 슬래시

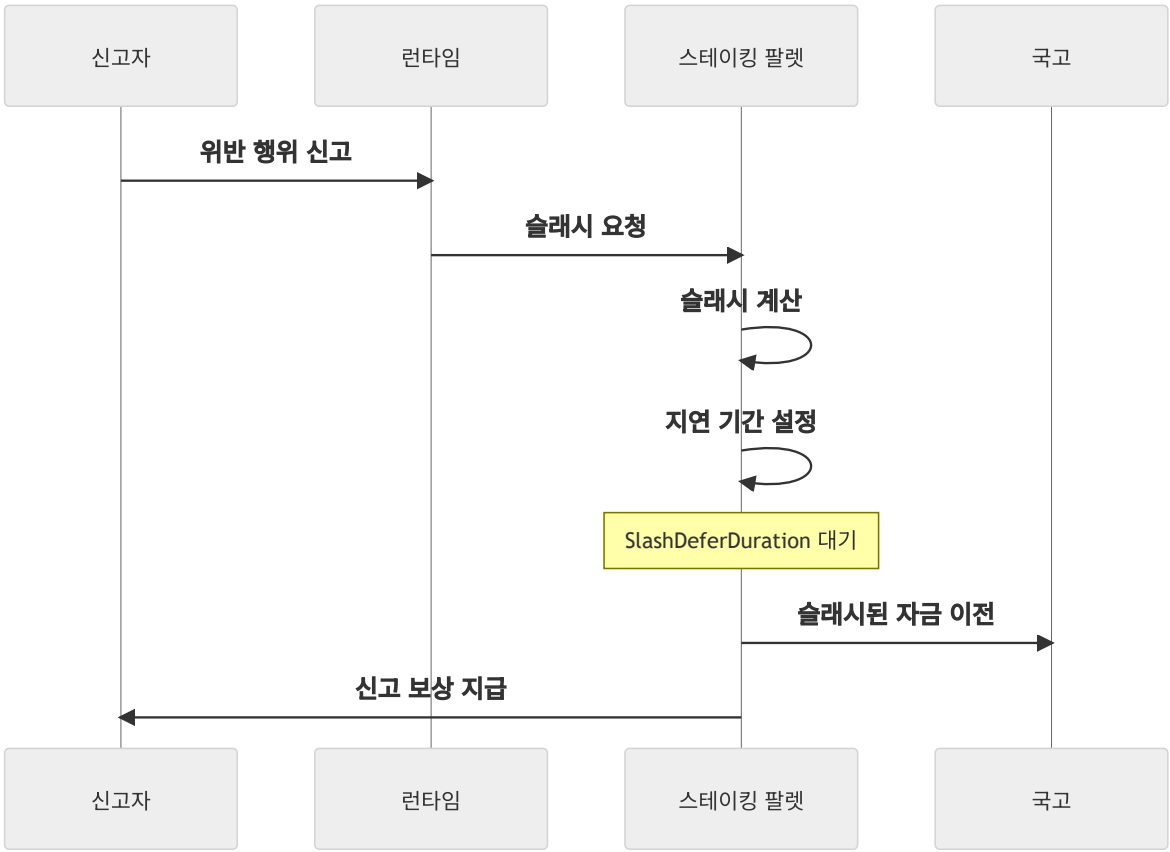
슬래싱 메커니즘

1. 슬래시 계산

```
// 슬래시 비율 계산
pub fn compute_slash_amount(
stake: Balance,
slash_fraction: Perbill,
exposure: &Exposure<AccountId, Balance>,
) -> Balance {
slash_fraction * stake
```

```
}
```

2. 슬래시 적용 과정



3. 슬래시 취소 메커니즘

```
# 거버넌스를 통한 슬래시 취소

polkadot-js-api  tx.staking.cancelDeferredSlash(era,  slashIndices)
```

검증자 성능 모니터링 및 최적화

1. 핵심 성능 지표

블록 생성 메트릭

```
# 블록 생성 성공률
polkadot_block_produced_total
polkadot_block_production_delay_seconds

# 블록 가져오기 성능
polkadot_import_queue_blocks_submitted
polkadot_import_queue_blocks_processed
```

네트워크 메트릭

```
# 피어 연결 상태
polkadot_sub_libp2p_peers_count
polkadot_sub_libp2p_connections_opened_total
polkadot_sub_libp2p_connections_closed_total

# 대역폭 사용량
polkadot_sub_libp2p_network_bytes_total
polkadot_sub_libp2p_notifications_total
```

파라체인 검증 메트릭

```
# 파라체인 검증 성능
```

```
polkadot_parachain_candidate_validation_seconds

polkadot_parachain_availability_recovery_time_total

polkadot_parachain_approval_checking_finalized_total
```

2. Grafana 대시보드 설정

검증자 대시보드 구성

```
{
"dashboard": {
"title": "Polkadot Validator Dashboard",
"panels": [
{
"title": "Block Production",
"targets": [
{
"expr": "rate(polkadot_block_produced_total[5m])",
"legendFormat": "Blocks/sec"
}
]
},
{
"title": "Peer Connections",
"targets": [
```

```
{
"expr": "polkadot_sub_libp2p_peers_count",
"legendFormat": "Connected Peers"
}
]
},
{
"title": "Finality Lag",
"targets": [
{
"expr": "polkadot_block_height{status=\"best\"} - polkadot_block_height{status=
"legendFormat": "Finality Lag"
}
]
}
]
}
}
```

3. 성능 최적화 전략

하드웨어 최적화

```
# CPU 성능 모드 설정
```

```
echo  performance | sudo  tee  /sys/devices/system/cpu/cpu*/cpufreq/scaling_gove


# 네트워크 버퍼 크기 증가
echo  'net.core.rmem_max = 134217728' >> /etc/sysctl.conf
echo  'net.core.wmem_max = 134217728' >> /etc/sysctl.conf
echo  'net.ipv4.tcp_rmem = 4096 87380 134217728' >> /etc/sysctl.conf
echo  'net.ipv4.tcp_wmem = 4096 65536 134217728' >> /etc/sysctl.conf


# 파일 디스크립터 한도 증가
echo  'polkadot soft nofile 65536' >> /etc/security/limits.conf
echo  'polkadot hard nofile 65536' >> /etc/security/limits.conf
```

노드 설정 최적화

```
# 워커 프로세스 수 최적화
polkadot \
--validator \
--execute-workers-max-num  8 \
--prepare-workers-hard-max-num 4 \
--prepare-workers-soft-max-num  2


# 데이터베이스 최적화
polkadot \
```

```
--validator \
--database  paritydb \
--db-cache 2048


# 네트워크 최적화
polkadot \
--validator \
--max-parallel-downloads  8 \
--in-peers 100 \
--out-peers  100
```

4. 알림 및 자동화

Prometheus 알림 규칙

```yaml
# validator-alerts.yml
groups:
- name: validator.rules
rules:
- alert: ValidatorDown
expr: up{job="polkadot-validator"} == 0
for: 1m
labels:
severity: critical
```

```
annotations:
summary: "Validator node is down"
- alert: HighFinalityLag
expr: polkadot_block_height{status="best"} - polkadot_block_height{status="final
for: 5m
labels:
severity: warning
annotations:
summary: "High finality lag detected"
- alert: LowPeerCount
expr: polkadot_sub_libp2p_peers_count < 10
for: 5m
labels:
severity: warning
annotations:
summary: "Low peer count"
```

자동 재시작 스크립트

```
#!/bin/bash
# validator-monitor.sh

VALIDATOR_RPC="http://localhost:9933"
```

```
LOG_FILE="/var/log/polkadot-monitor.log"


check_validator_health() {
# RPC 응답 확인
if ! curl -s -f "$VALIDATOR_RPC/health" > /dev/null; then
echo "$(date): Validator RPC not responding" >> "$LOG_FILE"
return 1
fi
# 동기화 상태 확인
local sync_state=$(curl -s -H "Content-Type: application/json" \
-d '{"id":1, "jsonrpc":"2.0", "method": "system_syncState"}' \
"$VALIDATOR_RPC" | jq -r '.result.currentBlock')
if [ "$sync_state" == "null" ]; then
echo "$(date): Validator not syncing" >> "$LOG_FILE"
return 1
fi
return 0
}


# 5분마다 상태 확인
while true; do
if ! check_validator_health; then
echo "$(date): Restarting validator service" >> "$LOG_FILE"
```

```
systemctl restart polkadot-validator
sleep 60 # 재시작 후 1분 대기
fi
sleep 300 # 5분 대기
done
```

검증자 운영 베스트 프랙티스

1. 보안 관리

- **키 분리**: 스테이킹 키와 세션 키 분리 보관
- **콜드 스토리지**: 스테이킹 키는 오프라인 환경에서 관리
- **정기 키 교체**: 세션 키 정기적 교체
- **방화벽 설정**: 필요한 포트만 개방

2. 고가용성 구성

- **이중화**: 메인/백업 노드 구성
- **자동 페일오버**: 장애 시 자동 전환
- **모니터링**: 24/7 모니터링 체계 구축
- **백업**: 정기적 데이터 백업

3. 성능 관리

- **리소스 모니터링**: CPU, 메모리, 디스크, 네트워크 사용량 추적
- **로그 분석**: 정기적 로그 분석 및 이상 징후 감지
- **업데이트 관리**: 노드 소프트웨어 정기 업데이트

- **용량 계획**: 미래 확장성 고려한 용량 계획

이 문서는 Polkadot 검증자 시스템의 전반적인 이해와 실제 운영에 필요한 실용적인 가이드를 제공합니다. 검증자 운영은 높은 기술적 전문성과 지속적인 관리가 필요한 작업이므로, 충분한 준비와 테스트를 거쳐 진행하는 것이 중요합니다.